

Team Review Report: Parallel Implementation Plan for Random Projections and Low-Rank Structure in Financial Return Data

Vishal Jha, Adel Sahuc
NYU MS Computer Science

Primary Paper: Michael B. Cohen, Thathachar S. Jayram, and Jelani Nelson.
Simple Analyses of the Sparse Johnson–Lindenstrauss Transform. SOSA 2018.

Contents

1	Executive Summary	2
2	Final Configuration for Team Review	3
3	Research Framing	4
3.1	Primary Paper Context	4
3.2	Why Financial Return Data Is a Meaningful Testbed	5
3.3	Core Research Questions	5
4	Data Design	6
4.1	Real Financial Dataset	6
4.2	Data Source	6
4.3	Stock Selection Rules	7
4.3.1	Layer 1: Candidate Pool	7
4.3.2	Layer 2: Ranking Metric	7
4.3.3	Layer 3: Elimination Pipeline	7
4.3.4	Diagnostic Output	8
4.3.5	Why No Reserve Replacement	9
4.4	Date Range	9
4.5	Return Matrix Definition	9
4.6	Interpretation of a Data Point	9

4.7	Missing Data Policy	10
4.8	Standardization	10
4.9	Train/Test Split	10
5	COVID and Market-Stress Evaluation Design	10
5.1	Motivation	10
5.2	COVID Window Definition	11
5.3	COVID Evaluation Slices	11
5.4	COVID-Specific Metrics	11
5.5	No-Leakage COVID Diagnostic Protocol	12
6	Method Design	12
6.1	Overview	12
6.2	Raw Baseline	13
6.3	Dense Gaussian Johnson–Lindenstrauss Projection	13
6.4	Sparse Johnson–Lindenstrauss Projection	14
6.5	PCA / Truncated SVD	14
6.6	JL Parameter Grid Search	14
6.6.1	Grid Search Outputs	15
6.7	Fairness Rules	15
7	Evaluation Metrics	15
7.1	Metric 1: Pairwise Distance Distortion	15
7.2	Metric 2: Nearest-Neighbor Preservation	16
7.3	Metric 3: Clustering Stability	16
7.4	Metric 4: Unusual Market Day Preservation	17
7.5	Metric 5: Runtime and Sparsity	17
8	Synthetic Factor-Model Experiment	18
8.1	Purpose	18
8.2	Synthetic Data Model	18
8.3	Synthetic Experimental Grid	19
8.4	Synthetic Experiment Diagram	19
9	Parallel Execution Plan	20

9.1	Why Parallelization Is Needed	20
9.2	Recommended Backend Strategy	20
9.3	Parallelization Unit	20
9.4	Parallel Execution Diagram	21
9.5	Required Backend Interface	22
9.6	Ray-Style Pseudocode	22
9.7	Parallelization Risks	22
10	Market Regime Analysis	23
10.1	Purpose	23
10.2	Market Return and Rolling Volatility	23
10.3	Regime-Specific Evaluation	23
11	Required Figure Plan	24
11.1	Purpose of the Expanded Figure Set	24
11.2	Figure Generation Principles	24
11.3	Minimum Figure Catalogue	24
11.4	Main-Report vs. Appendix Figures	26
12	Expected Results and Interpretation	27
12.1	Expected Result 1: PCA Dominates Under Strong Low-Rank Structure	27
12.2	Expected Result 2: Sparse JL Becomes Competitive as k Increases	27
12.3	Expected Result 3: Noise Weakens PCA’s Advantage	27
12.4	Expected Result 4: COVID and High-Volatility Regimes May Be Harder to Preserve	28
13	Implementation Output Contract	28
13.1	Minimum Required Tables	29
13.2	Minimum Required Figures	30
14	Implementation Phases	30
14.1	Phase 1: Finalize Design	30
14.2	Phase 2: Data Pipeline	30
14.3	Phase 3: Compression Methods	31
14.4	Phase 4: Metrics	31
14.5	Phase 5: Parallel Experiment Runner	32

14.6 Phase 6: Real-Data Experiments	32
14.7 Phase 7: COVID Experiments	32
14.8 Phase 8: Synthetic Experiments	32
14.9 Phase 9: Figure Generation	33
14.10Phase 10: Analysis and Report	33
15 Risks and Mitigations	33
16 Team Decisions Required Before Implementation	35
17 Proposed Final Report Structure	37
18 Definition of Success	39
19 Reference	39

1 Executive Summary

This document defines the proposed implementation plan for our project on random projections for cross-sectional compression of financial asset return data. The project is motivated by the SOSA 2018 paper *Simple Analyses of the Sparse Johnson–Lindenstrauss Transform* by Cohen, Jayram, and Nelson. That paper studies sparse Johnson–Lindenstrauss transforms, which are randomized linear maps that preserve Euclidean geometry while using substantially fewer nonzero entries than dense random projections.

Our project asks whether this sparse, data-independent compression perspective remains useful in a structured financial setting. Financial asset returns are not arbitrary high-dimensional vectors. They are strongly correlated, affected by common market and sector forces, and often approximately governed by a small number of latent factors. This suggests that financial return matrices may have approximate low-rank structure. PCA or truncated SVD should exploit this structure directly, while sparse JL and dense Gaussian random projections are oblivious to it.

The central research question is:

When does data-independent sparse JL compression preserve the geometry of factor-structured financial return data, and when does data-adaptive PCA dominate?

The proposed empirical study compares four representations:

1. Raw uncompressed return vectors.
2. Dense Gaussian Johnson–Lindenstrauss projections.
3. Sparse Johnson–Lindenstrauss projections.
4. PCA / truncated SVD.

The primary real-data experiment uses a cross-sectional equity return matrix built from S&P 500 constituents (today’s membership, ~500 tickers), sourced from a pinned public Kaggle dataset for reproducibility. Each row is one trading day and each column is one asset; each row is treated as a market-state vector. The project evaluates whether compressed representations preserve distances, nearest neighbors, clustering structure, and unusual market days.

The secondary controlled experiment uses synthetic factor-model data of the form

$$X = FB^{\top} + E,$$

where F contains latent factor returns, B contains asset factor loadings, and E is idiosyncratic noise. This synthetic experiment is critical because it lets us directly vary the true factor rank and noise level, allowing us to isolate when PCA should beat JL and when sparse JL may remain competitive.

This updated plan adds two major implementation improvements:

1. **Parallel execution plan:** The experiment grid is large enough that a purely linear implementation may be slow. We therefore design the system so that experiment configurations can run independently using a backend abstraction. The preferred backend is Ray when available, with a local sequential fallback for reproducibility.

2. **Expanded figure plan:** The final report should not rely on only a few aggregate plots. The implementation must generate at least 40 figures covering raw data diagnostics, PCA structure, JL grid-search behavior, real-data metrics, COVID-specific diagnostics, synthetic factor-model experiments, runtime/memory tradeoffs, and robustness checks.

2 Final Configuration for Team Review

The following configuration is the proposed default. Once approved, this should be frozen and converted into an implementation plan for the coding agent.

Table 1: Proposed Experimental Configuration

Component	Decision
Asset universe	S&P 500 today’s constituents (~ 500 names); survivorship-biased, declared in limitations
Data source	Kaggle: <code>andrewmvd/sp-500-stocks</code> (Larxel), pinned dataset version; yfinance fallback if unavailable
Selection framework	Three-layer pipeline: (1) S&P 500 candidate pool, (2) rank by avg dollar volume, (3) elimination pipeline (data integrity + anomaly filters), then take <code>top_100</code> , <code>top_200</code> , or <code>all_survivors</code>
Date range	2014-01-01 to 2024-12-31
Return type	Daily log returns
Data matrix	Rows = trading days, columns = assets
Missing data policy	No imputation in the primary experiment; tickers with missing values are dropped at Stage 1 of the elimination pipeline; no reserve replacement
Preprocessing	Z-score each asset using training-set statistics only
Train/test split	Chronological 70/30 split for primary evaluation
Core methods	Raw, dense Gaussian JL, sparse JL, PCA / truncated SVD
Primary target dimensions	$k \in \{5, 10, 20, 30, 50, 100\}$
Sparse JL sparsity	Primary grid $s \in \{1, 3, 5\}$; primary value $s = 3$
Random seeds	10 seeds for randomized JL methods
Real-data metrics	Distance distortion, nearest-neighbor overlap, clustering ARI, anomaly recall, runtime, memory
COVID-specific evaluation	Full sample including COVID, full sample excluding COVID, COVID-stress-only window
Synthetic model	$X = FB^\top + E$
Synthetic ranks	$r \in \{3, 5, 10, 20\}$
Synthetic noise levels	$\sigma \in \{0.1, 0.5, 1.0\}$
Parallel execution	Ray backend preferred for grid experiments; local backend required as fallback
Minimum figure count	At least 40 generated figures
Primary story	PCA dominates under strong low-rank structure; sparse JL becomes competitive when structure weakens, noise increases, k increases, or efficiency matters

3 Research Framing

3.1 Primary Paper Context

The primary paper, *Simple Analyses of the Sparse Johnson–Lindenstrauss Transform*, studies sparse variants of the Johnson–Lindenstrauss transform. The classical Johnson–Lindenstrauss lemma says that a set of high-dimensional points can be embedded into a much lower-dimensional Euclidean space while approximately preserving pairwise distances. Dense Gaussian random pro-

jections are a common way to achieve this, but dense projections can be computationally expensive when the original dimension is large.

Sparse JL transforms aim to preserve the same basic geometric behavior while reducing computational cost. Instead of multiplying by a dense random matrix, the input is multiplied by a sparse random matrix with relatively few nonzero entries. The SOSA paper is important because it provides simple analyses of these sparse transforms and helps explain why sparse random projections can retain useful distance-preservation guarantees.

3.2 Why Financial Return Data Is a Meaningful Testbed

Financial return data is a structured domain. Cross-sectional equity returns are affected by common forces such as:

- market-wide movements,
- sector-level effects,
- style factors,
- macroeconomic shocks,
- liquidity and volatility regimes.

This means the data may lie close to a low-dimensional factor space. A simplified factor model has the form

$$x_t = Bf_t + \epsilon_t,$$

where $x_t \in \mathbb{R}^N$ is the return vector across N assets at time t , $B \in \mathbb{R}^{N \times r}$ contains factor loadings, $f_t \in \mathbb{R}^r$ contains factor returns, and ϵ_t is idiosyncratic noise.

This creates a natural tension:

- PCA is data-adaptive and should exploit low-rank factor structure.
- Sparse JL is data-independent and does not explicitly learn the factor space.
- Dense Gaussian JL is also data-independent but less sparse and more computationally expensive.

The project therefore tests whether sparse JL remains empirically useful when the data has financial structure that PCA is designed to exploit.

3.3 Core Research Questions

The implementation should directly support the following three research questions:

1. **Generalization of sparse JL intuition:** To what extent do the structure-preservation properties of sparse JL transforms remain meaningful when applied to highly correlated financial asset return data rather than generic high-dimensional vectors?
2. **Low-rank factor structure vs. sparse JL:** When asset returns lie near an approximate low-rank factor model, when do sparse random projections preserve market geometry nearly as

well as PCA, and when does the low-rank structure make PCA fundamentally superior at the same compressed dimension?

3. **Role of market structure:** Which properties of financial data, such as factor strength, cross-sectional correlation, volatility regime, and COVID-style stress periods, most strongly affect whether sparse JL projections preserve nearest neighbors, clustering structure, and unusual market states?

4 Data Design

4.1 Real Financial Dataset

The real dataset consists of daily adjusted close prices for U.S. equities in the S&P 500. The team adopts the following data plan:

Asset universe: S&P 500 constituents (current membership, ~ 500 names). The full S&P 500 is preferred over the smaller S&P 100 because (a) it provides a higher-dimensional cross-section ($N \approx 500$), making the compression research question more interesting, and (b) historical S&P 500 data is far better supported by public sources than S&P 100 data.

Survivorship bias acknowledgement: Using today’s S&P 500 backfilled to 2014 is survivorship-biased. Constituents that were in the S&P 500 in 2014 but were later acquired, bankrupt, or demoted (e.g., Time Warner, Sprint, Allergan) are excluded. This is a known limitation. The project’s research question concerns compression geometry on real structured financial data, not trading-strategy realism, so this bias is acceptable as long as it is clearly stated. Survivorship bias is addressed in the limitations section of the final report (see Section 15).

4.2 Data Source

Primary source: Kaggle dataset *S&P 500 stocks (daily updated)* maintained by user **andrewmvd** (Larxel). Available at:

<https://www.kaggle.com/datasets/andrewmvd/sp-500-stocks>

The implementation must:

1. Pin a specific dataset version (record the Kaggle version number and download date in `configs/experiment.conf`)
2. Commit the downloaded CSV files to `data/raw/` so the project is reproducible without re-downloading.
3. Validate the CSV against the no-imputation policy in Section 4.7.

Why a public CSV instead of a live API: Live APIs (yfinance, Tiingo, Alpha Vantage) introduce reproducibility risk because returned values depend on download timing, vendor adjustment choices, and API stability. A pinned Kaggle CSV gives a frozen, version-controlled input that any reviewer can re-run and obtain identical results.

Why Kaggle is acceptable for a research project: The S&P 500 historical price dataset is heavily used by the academic and student community. It is not the canonical source (CRSP via

WRDS is), but for a research project on compression geometry it is a defensible trade-off given the lack of free WRDS access. The bias and quality trade-offs are documented in Section 15.

Fallback source: If the Kaggle dataset becomes unavailable or fails validation, the data pipeline must support a yfinance-based fallback that reproduces the same schema. The fallback uses the same fixed ticker list (today’s S&P 500), scraped from Wikipedia’s *List of S&P 500 companies* page.

Validation cross-check (recommended): For a small random sample of tickers (e.g., 5–10 names), the pipeline should compare Kaggle adjusted-close values against a second source (yfinance or Stooq) for a handful of dates. Material disagreements should be logged. This is a sanity check, not a full audit.

4.3 Stock Selection Rules

The pipeline applies a deterministic three-layer framework to convert the S&P 500 candidate pool into a final analysis universe. The three layers separate *candidate sourcing* from *ranking* from *elimination*, so each can be reasoned about independently.

4.3.1 Layer 1: Candidate Pool

The candidate pool is today’s S&P 500 constituents (~500 names, with ~503 actual rows due to dual-class shares such as GOOG/GOOGL). The list is loaded from the pinned Kaggle dataset’s ticker manifest. Output: `data/processed/candidate_universe.csv`.

4.3.2 Layer 2: Ranking Metric

Tickers in the candidate pool are ranked by **average daily dollar volume** over the full study window:

$$\text{Rank}_i = \frac{1}{T} \sum_{t=1}^T P_{t,i} \cdot V_{t,i},$$

where $P_{t,i}$ is the adjusted close price and $V_{t,i}$ is the daily share volume.

This metric was chosen because:

- It composes naturally with the elimination rules: stocks that delisted mid-period get zero volume after delisting and rank low automatically.
- It is a better proxy for “stocks that traded actively and reliably” than market capitalization, which is dominated by a handful of mega-caps.
- Both inputs (P and V) are available in any standard data source, including the Kaggle CSV.

The ranking is computed once over the full window and frozen. Output: `data/processed/ranked_universe.csv`.

4.3.3 Layer 3: Elimination Pipeline

The following filters are applied in order. Each filter logs the number of tickers it drops and the list of dropped ticker symbols.

Stage 1 – Data integrity filter. Drop a ticker if any of the following hold:

- Any NaN in adjusted close inside [2014-01-01, 2024-12-31].
- The ticker was delisted before 2024-12-31 (last trade date earlier than the window end). This is the bankrupt / acquired / removed-from-index case.
- Fewer than 95% of expected trading days are present.
- Any single-day return $|r| > 0.5$ that does not correspond to a known corporate action (treated as a data error).

Stage 2 – Anomaly filter. Drop a ticker if any of the following hold:

- More than N_{flat} consecutive days of exactly zero return (likely halted or stale data); recommended $N_{\text{flat}} = 10$.
- Adjusted close exactly constant for more than 5 consecutive days (likely a data error).

Note: No liquidity floor is applied. The ranking metric already pushes illiquid names down the list, and an explicit dollar-volume threshold would impose an arbitrary cutoff without scientific justification for a compression-geometry study.

Stage 3 – Selection. From the surviving tickers, take the top- N by the ranking metric. Supported choices:

- `top_100` – top 100 by average dollar volume.
- `top_200` – top 200 by average dollar volume.
- `all_survivors` – every ticker that passed Stages 1–2.

The chosen slice is recorded in `configs/experiment_config.yaml` under the `universe` key. Running the same pipeline at multiple slices ($N = 100, 200, \text{all}$) provides a natural sensitivity-to- N analysis at no extra design cost.

4.3.4 Diagnostic Output

After the pipeline runs, `data/processed/preprocessing_report.json` must include:

- Kaggle dataset version pinned and download date,
- candidate pool size,
- Stage 1 drop count and dropped ticker list,
- Stage 2 drop count and dropped ticker list,
- total survivors,
- composition of `top_100`, `top_200`, and `all_survivors` slices,
- ranking metric values for the chosen slice,
- confirmation that the final return matrix contains no missing values.

This report drives the data-cleaning table in the final report.

4.3.5 Why No Reserve Replacement

Earlier versions of this proposal called for a reserve list of “next-ranked S&P 500 names” to backfill the universe to a fixed target N . With the three-layer framework this is unnecessary: the ranking provides a natural ordering, so “top 100” is well-defined regardless of how many candidates failed the elimination filters. If fewer than N tickers survive Stages 1–2, that itself is a finding worth reporting – not something to paper over with replacements.

4.4 Date Range

The proposed date range is:

2014-01-01 to 2024-12-31.

This gives approximately 10 years of daily trading data. The date range includes multiple market regimes, including:

- normal low-volatility periods,
- the COVID-19 shock,
- post-COVID high-volatility periods,
- inflation and rate-hike periods.

4.5 Return Matrix Definition

Let $P_{t,i}$ be the adjusted close price of asset i on trading day t . We compute daily log returns:

$$X_{t,i} = \log \left(\frac{P_{t,i}}{P_{t-1,i}} \right).$$

The final real-data matrix is:

$$X \in \mathbb{R}^{T \times N},$$

where:

- T is the number of trading days after cleaning,
- N is the number of assets retained after filtering,
- row $X_t \in \mathbb{R}^N$ is one market-state vector.

4.6 Interpretation of a Data Point

Each row of X is one market snapshot:

$$X_t = [r_{t,1} \quad r_{t,2} \quad \cdots \quad r_{t,N}].$$

The geometry being preserved is the geometry between trading days, not the geometry between individual assets. If two days have similar cross-sectional return behavior in raw space, then a good compression method should keep them close in compressed space.

4.7 Missing Data Policy

The primary experiment uses a **no-imputation** policy. Tickers with any missing adjusted-close values inside the target date range are dropped at Stage 1 of the elimination pipeline (Section 4.3). No forward-filling or interpolation is applied to the primary return matrix.

Note: A secondary sensitivity experiment may use forward-fill or interpolation if desired, but it must not be the primary reported result.

4.8 Standardization

The primary experiment uses asset-wise z-scoring based only on training-set statistics.

Let X_{train} be the training matrix. For each asset i , compute:

$$\mu_i = \frac{1}{T_{\text{train}}} \sum_{t \in \text{train}} X_{t,i},$$
$$\sigma_i = \sqrt{\frac{1}{T_{\text{train}} - 1} \sum_{t \in \text{train}} (X_{t,i} - \mu_i)^2}.$$

Then standardize:

$$\tilde{X}_{t,i} = \frac{X_{t,i} - \mu_i}{\sigma_i}.$$

The same training-set μ_i and σ_i must be used to transform both train and test data. This avoids leakage.

4.9 Train/Test Split

Use a chronological 70/30 split for the primary evaluation:

$$X = \begin{bmatrix} X_{\text{train}} \\ X_{\text{test}} \end{bmatrix}.$$

The split must not be randomized because financial time series have temporal structure. PCA is fit only on X_{train} and then applied to X_{test} .

5 COVID and Market-Stress Evaluation Design

5.1 Motivation

The team explicitly wants the final analysis to zoom in on COVID as a special market-stress period. This is important because a method that performs well in ordinary regimes may fail during sudden market-wide shocks.

The project should therefore produce three separate result sets:

1. **Including-COVID results:** evaluation over the full available evaluation horizon, including COVID.
2. **Excluding-COVID results:** evaluation over the same horizon after removing COVID-stress days.
3. **COVID-only results:** evaluation only on the COVID-stress window.

5.2 COVID Window Definition

Use the following primary COVID-stress window:

$$\text{COVID-stress window} = 2020-02-19 \text{ to } 2020-04-30.$$

This window is intended to capture the sharp market crash and immediate stress period.

Optionally, the implementation may also produce a broader COVID-year diagnostic:

$$\text{COVID-year diagnostic window} = 2020-01-01 \text{ to } 2020-12-31.$$

The primary final report should focus on the stress window. The broader COVID-year result can be placed in the appendix.

5.3 COVID Evaluation Slices

Define three evaluation slices:

1. **Full-Including-COVID:**

$$\mathcal{T}_{\text{full}} = \{t : 2014-01-01 \leq t \leq 2024-12-31\}.$$

2. **Full-Excluding-COVID:**

$$\mathcal{T}_{\text{exCOVID}} = \mathcal{T}_{\text{full}} \setminus \{t : 2020-02-19 \leq t \leq 2020-04-30\}.$$

3. **COVID-Only:**

$$\mathcal{T}_{\text{COVID}} = \{t : 2020-02-19 \leq t \leq 2020-04-30\}.$$

5.4 COVID-Specific Metrics

For each of the three slices, compute:

- pairwise distance distortion,
- nearest-neighbor preservation,
- clustering stability,
- anomaly recall,
- anomaly-score correlation,

- runtime and memory where applicable.

For COVID-only analysis, also compute:

- top-10 most unusual days in raw space,
- rank of those days under each compressed representation,
- overlap between raw top-10 unusual days and compressed top-10 unusual days,
- distance-distortion distribution during COVID-only window,
- nearest-neighbor degradation relative to non-COVID periods.

5.5 No-Leakage COVID Diagnostic Protocol

Because COVID occurs before the final 30% of the 2014–2024 date range, the standard chronological 70/30 test split may not isolate COVID as an out-of-sample event. Therefore, the implementation should use two protocols:

1. **Primary chronological protocol:** Train on first 70%, test on final 30%. This is the main generalization protocol.
2. **COVID diagnostic protocol:** Fit standardization parameters and PCA on pre-COVID data only:

$$\mathcal{T}_{\text{preCOVID-train}} = \{t : 2014-01-01 \leq t \leq 2019-12-31\}.$$

Then evaluate on:

- non-COVID control windows,
- COVID-only stress window,
- post-COVID windows.

This second protocol is essential for a fair stress-test interpretation.

6 Method Design

6.1 Overview

The study compares raw data, dense Gaussian JL, sparse JL, and PCA at the same target dimensions.

The core target-dimension grid is:

$$k \in \{5, 10, 20, 30, 50, 100\}.$$

This grid should be used for all main tables and main-report figures.

Each method maps a market-state vector from $\mathbb{R}^N$ to $\mathbb{R}^k$, except the raw baseline, which remains in $\mathbb{R}^N$.

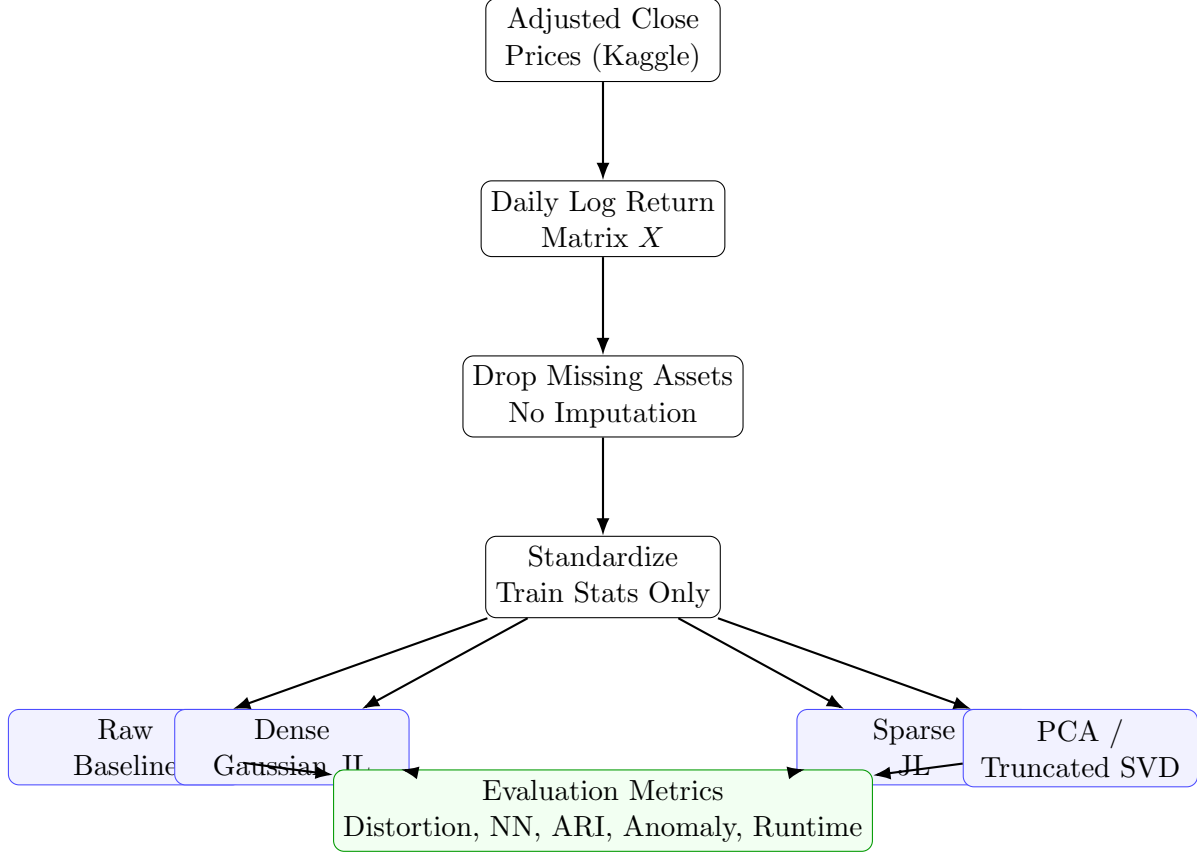


Figure 1: Real-data experimental pipeline with strict no-imputation cleaning.

6.2 Raw Baseline

The raw baseline uses the standardized return vectors directly:

$$Z_{\text{raw}} = \tilde{X}.$$

This is not a compressed representation. It serves as the reference geometry against which all compressed methods are evaluated.

6.3 Dense Gaussian Johnson–Lindenstrauss Projection

For dense Gaussian JL, generate a projection matrix:

$$A \in \mathbb{R}^{N \times k},$$

with entries:

$$A_{ij} \sim \mathcal{N}\left(0, \frac{1}{k}\right).$$

The compressed representation is:

$$Z_{\text{gaussian}} = \tilde{X}A.$$

For each target dimension k , the method should be run over 10 random seeds. The report should show the mean and standard deviation of each metric across seeds.

6.4 Sparse Johnson–Lindenstrauss Projection

For sparse JL, generate a sparse projection matrix:

$$S \in \mathbb{R}^{N \times k}.$$

For each input coordinate j , choose s output coordinates uniformly at random and assign random signs. The nonzero entries are scaled as:

$$S_{j,\ell} \in \left\{ -\frac{1}{\sqrt{s}}, +\frac{1}{\sqrt{s}} \right\}.$$

All other entries are zero.

The compressed representation is:

$$Z_{\text{sparse}} = \tilde{X}S.$$

The core sparse grid is:

$$s \in \{1, 3, 5\}.$$

The primary sparse JL setting should be $s = 3$.

6.5 PCA / Truncated SVD

PCA is the main data-adaptive low-rank baseline. Fit PCA on training data only:

$$\text{PCA}_k \leftarrow \text{fit}(X_{\text{train}}).$$

Then transform train and test:

$$Z_{\text{train,pca}} = X_{\text{train}}V_k,$$

$$Z_{\text{test,pca}} = X_{\text{test}}V_k,$$

where $V_k \in \mathbb{R}^{N \times k}$ contains the top k principal directions.

PCA should be evaluated at the same k values as JL methods.

6.6 JL Parameter Grid Search

The JL parameter grid is the cross-product of the core k grid and the core sparsity grid:

$$k \in \{5, 10, 20, 30, 50, 100\}, \quad s \in \{1, 3, 5\}.$$

Only valid sparse JL combinations should be run:

$$s \leq k.$$

6.6.1 Grid Search Outputs

The JL grid search should produce:

- best sparse JL configuration by distance distortion,
- best sparse JL configuration by nearest-neighbor overlap,
- best sparse JL configuration by anomaly recall,
- runtime-memory tradeoff frontier,
- comparison against dense Gaussian JL,
- comparison against PCA at the same k .

6.7 Fairness Rules

To make the comparison fair:

1. All compressed methods must use the same target dimensions.
2. PCA must be fit on training data only.
3. Standardization must use training statistics only.
4. Random projection methods must be averaged across 10 random seeds.
5. Evaluation should be performed primarily on the test set.
6. The raw test-set geometry is the reference geometry.
7. COVID diagnostic experiments must clearly state whether PCA was fit on the primary chronological training set or the pre-COVID training set.

7 Evaluation Metrics

7.1 Metric 1: Pairwise Distance Distortion

For sampled pairs of test days (i, j) , compute the raw-space distance:

$$d_{ij} = \|X_i - X_j\|_2.$$

For a compressed representation Z , compute:

$$\hat{d}_{ij} = \|Z_i - Z_j\|_2.$$

The multiplicative distortion is:

$$\rho_{ij} = \frac{\hat{d}_{ij}}{d_{ij}}.$$

A perfect distance-preserving embedding would have:

$$\rho_{ij} = 1.$$

Report the following:

- mean absolute distortion:

$$\frac{1}{M} \sum_{(i,j)} |\rho_{ij} - 1|,$$

- median distortion,
- 95th percentile absolute distortion,
- distribution plots of ρ_{ij} ,
- COVID vs. non-COVID distortion comparison.

This is the most direct metric connected to the Johnson–Lindenstrauss perspective.

7.2 Metric 2: Nearest-Neighbor Preservation

For each test day i , compute its m nearest neighbors in raw space:

$$\mathcal{N}_m^{\text{raw}}(i).$$

Then compute its m nearest neighbors in compressed space:

$$\mathcal{N}_m^{\text{comp}}(i).$$

The neighbor overlap is:

$$\text{NNOverlap}_m(i) = \frac{|\mathcal{N}_m^{\text{raw}}(i) \cap \mathcal{N}_m^{\text{comp}}(i)|}{m}.$$

Use:

$$m \in \{5, 10\}.$$

Report average top-5 and top-10 nearest-neighbor overlap across all test days and across COVID-specific slices.

7.3 Metric 3: Clustering Stability

The goal is to test whether compressed representations preserve market-state clusters.

Procedure:

1. Run KMeans clustering on raw test data.
2. Run KMeans clustering on compressed test data.
3. Compare cluster labels using Adjusted Rand Index.

Use cluster counts:

$$C \in \{3, 5, 8\}.$$

The main clustering metric is Adjusted Rand Index:

$$\text{ARI} \in [-1, 1],$$

where larger values indicate stronger agreement between clusterings.

Report:

- ARI by method and k ,
- average ARI across cluster counts,
- sensitivity to cluster count,
- COVID vs. non-COVID ARI.

7.4 Metric 4: Unusual Market Day Preservation

Define an anomaly score in raw space:

$$a_i = \|X_i\|_2.$$

A test day is considered unusual if its raw anomaly score lies in the top 5%:

$$i \in \mathcal{A}_{\text{raw}}.$$

For compressed representation Z , compute:

$$\hat{a}_i = \|Z_i\|_2.$$

Let $\mathcal{A}_{\text{comp}}$ be the top 5% of days according to compressed anomaly scores.

Report:

$$\text{Recall@5\%} = \frac{|\mathcal{A}_{\text{raw}} \cap \mathcal{A}_{\text{comp}}|}{|\mathcal{A}_{\text{raw}}|}.$$

Also report:

- precision@5%,
- anomaly-score correlation,
- top-10 unusual day overlap,
- COVID-only anomaly preservation.

7.5 Metric 5: Runtime and Sparsity

Since sparse JL is motivated by efficiency, the implementation should also record:

- projection matrix construction time,
- transform runtime,

- metric computation runtime,
- number of nonzero entries in the projection matrix,
- memory footprint of the projection matrix,
- total end-to-end experiment time.

This allows the final report to discuss whether sparse JL is practically attractive even if PCA performs better on some accuracy metrics.

8 Synthetic Factor-Model Experiment

8.1 Purpose

The synthetic experiment is essential for the low-rank factor structure question. Real financial data may be noisy and difficult to interpret. Synthetic factor data allows us to control:

- true factor rank,
- noise level,
- factor strength,
- dimensionality.

This gives a clean way to answer:

When data is strongly low-rank, does PCA clearly dominate sparse JL? As noise increases or the factor structure weakens, does sparse JL become more competitive?

8.2 Synthetic Data Model

Generate:

$$X = FB^\top + E,$$

where:

- $X \in \mathbb{R}^{T \times N}$ is the synthetic return matrix,
- $F \in \mathbb{R}^{T \times r}$ contains latent factor returns,
- $B \in \mathbb{R}^{N \times r}$ contains asset factor loadings,
- $E \in \mathbb{R}^{T \times N}$ is idiosyncratic noise,
- r is the true latent factor rank.

A simple implementation is:

$$F_{t,j} \sim \mathcal{N}(0, 1),$$

$$B_{i,j} \sim \mathcal{N}(0, 1),$$

$$E_{t,i} \sim \mathcal{N}(0, \sigma^2).$$

Then:

$$X = FB^\top + E.$$

8.3 Synthetic Experimental Grid

Use:

$$r \in \{3, 5, 10, 20\},$$

$$\sigma \in \{0.1, 0.5, 1.0\}.$$

For each (r, σ) pair, evaluate all methods at:

$$k \in \{5, 10, 20, 30, 50, 100\}.$$

The expected behavior is:

- When r is small and σ is low, PCA should perform extremely well.
- When σ increases, the advantage of PCA may shrink.
- When k is much larger than r , random projections may become more competitive.
- Sparse JL may provide a useful efficiency tradeoff if its metrics are close to dense JL or PCA.

8.4 Synthetic Experiment Diagram

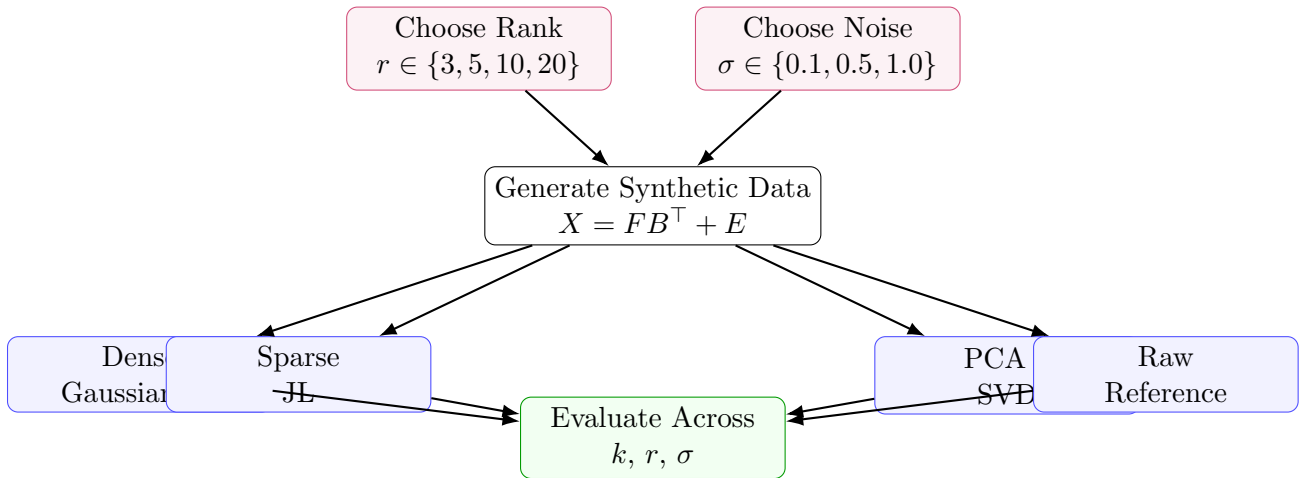


Figure 2: Synthetic factor-model experiment.

9 Parallel Execution Plan

9.1 Why Parallelization Is Needed

The experiment grid is large. A naive linear implementation would run:

- multiple methods,
- multiple target dimensions,
- multiple sparse JL sparsity values,
- multiple random seeds,
- multiple evaluation slices,
- multiple synthetic ranks,
- multiple synthetic noise levels,
- multiple metrics.

Most experiment configurations are embarrassingly parallel. For example, sparse JL with $(k = 20, s = 3, seed = 4)$ can be evaluated independently from sparse JL with $(k = 50, s = 5, seed = 7)$. Synthetic experiments are also independent across $(r, \sigma, k, s, seed)$.

Therefore, the implementation should be designed with a parallel execution backend.

9.2 Recommended Backend Strategy

The implementation should support two execution backends:

1. **Local backend:** runs experiments sequentially or with standard multiprocessing. This is required for debugging and reproducibility.
2. **Ray backend:** runs independent experiment configurations in parallel. This is preferred for the full grid search and synthetic experiments.

Ray is useful if:

- synthetic rank/noise experiments are included,
- COVID-specific slices are evaluated separately,
- repeated random seeds are used,
- figure generation is separated from metric computation.

Ray is not strictly necessary for a minimal version, but the code should be designed so that Ray can be enabled without rewriting the experiment logic.

9.3 Parallelization Unit

The atomic unit of parallel work should be an experiment configuration:

```

ExperimentConfig(
    dataset_type,
    evaluation_slice,
    method,
    k,
    sparse_s,
    seed,
    synthetic_rank,
    synthetic_noise,
    metric_bundle
)

```

Each worker should:

1. load or receive the prepared dataset,
2. construct the projection or fit PCA if needed,
3. transform the selected evaluation data,
4. compute the requested metrics,
5. return a structured result dictionary,
6. save intermediate outputs only if requested.

9.4 Parallel Execution Diagram

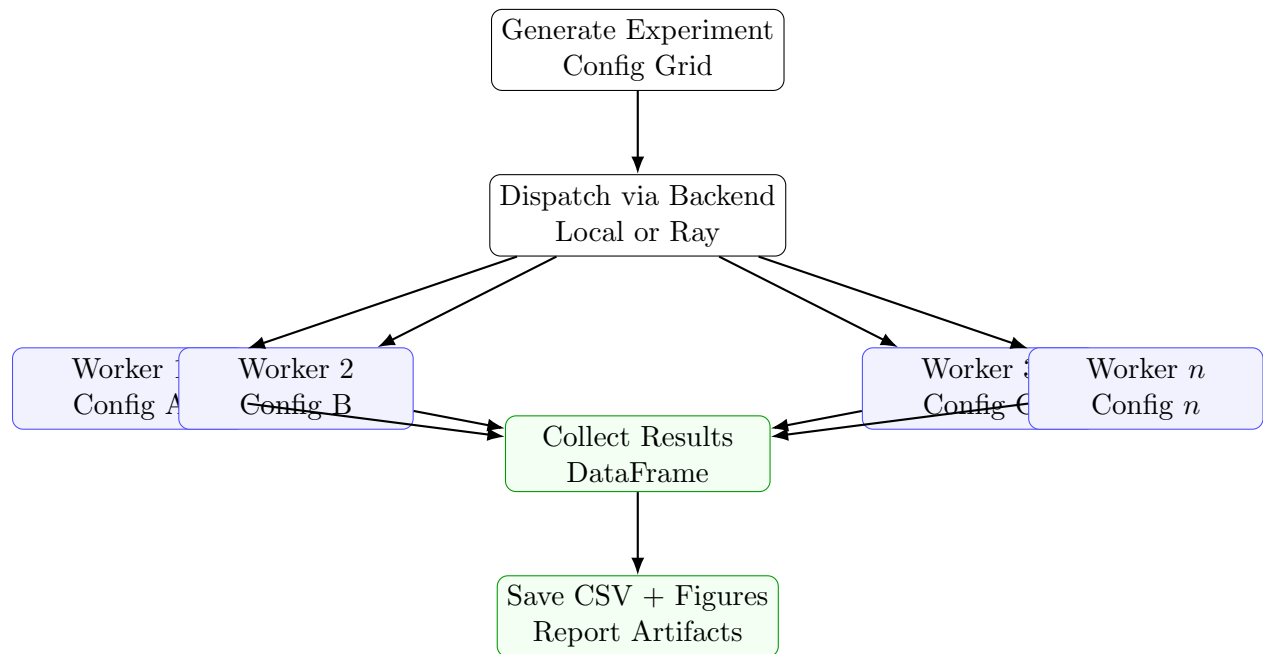


Figure 3: Parallel experiment execution design.

9.5 Required Backend Interface

The implementation should expose a backend-agnostic runner:

```
def run_experiment_grid(configs, backend="local", num_cpus=None):
    """
    Runs a list of ExperimentConfig objects.

    backend:
    "local" -> sequential or multiprocessing fallback
    "ray" -> Ray-based parallel execution

    Returns:
    pandas.DataFrame with one row per experiment result.
    """
```

The local and Ray backends should produce identical result schemas.

9.6 Ray-Style Pseudocode

```
@ray.remote
def run_single_config_remote(config, shared_data_ref):
    data = ray.get(shared_data_ref)
    return run_single_config(config, data)

def run_with_ray(configs, prepared_data, num_cpus=None):
    ray.init(num_cpus=num_cpus, ignore_reinit_error=True)
    shared_data_ref = ray.put(prepared_data)

    futures = [
        run_single_config_remote.remote(config, shared_data_ref)
        for config in configs
    ]

    results = ray.get(futures)
    return pd.DataFrame(results)
```

9.7 Parallelization Risks

Table 2: Parallelization Risks and Mitigations

Risk	Why It Matters	Mitigation
Ray overhead exceeds benefit	Small datasets may not need distributed execution	Keep local backend; use Ray only when grid is large
Random seeds inconsistent	Parallel execution can make randomness harder to reproduce	Store seed in every config and result row
Large data copied to workers	Memory overhead can increase	Use shared object store via <code>ray.put</code> ; avoid repeated copies

Risk	Why It Matters	Mitigation
Race conditions in output files	Multiple workers may write to same path	Workers return results; central process writes outputs
Difficult debugging	Parallel traces are harder to inspect	Debug first using local backend on small grid
Environment mismatch	Ray may not be installed on all machines	Make Ray optional dependency; local backend required

10 Market Regime Analysis

10.1 Purpose

Financial data changes across regimes. A method that works well in calm periods may fail during high-volatility periods. To address this, the real-data test set should be divided into volatility regimes.

10.2 Market Return and Rolling Volatility

Define the equal-weighted market return:

$$m_t = \frac{1}{N} \sum_{i=1}^N X_{t,i}.$$

Compute 20-day rolling volatility:

$$v_t = \text{std}(m_{t-19}, \dots, m_t).$$

Divide test days into three groups:

- low-volatility regime,
- medium-volatility regime,
- high-volatility regime.

The split can be based on volatility tertiles.

10.3 Regime-Specific Evaluation

For each volatility regime, recompute:

- pairwise distance distortion,
- nearest-neighbor preservation,
- anomaly recall,

- anomaly-score correlation.

This directly supports the third research question: whether volatility regimes affect the success or failure of sparse JL.

11 Required Figure Plan

11.1 Purpose of the Expanded Figure Set

The final report should include enough figures to make the data behavior and method comparisons interpretable. The goal is not to add plots for volume. The goal is to make every major conclusion visually defensible.

The implementation must generate at least 40 figures. The final written report can include the most important figures in the main body and move the rest to an appendix.

11.2 Figure Generation Principles

Every figure should satisfy the following rules:

1. The filename must be deterministic.
2. The plot title must identify the dataset slice, method family, and metric.
3. Axes must be labeled clearly.
4. If random seeds are involved, plot mean with variability bands or error bars.
5. The underlying CSV used for each figure must be saved.
6. Figures should be generated from saved result files, not directly from transient in-memory experiment state.

11.3 Minimum Figure Catalogue

Table 3: Required Figure Catalogue: At Least 40 Figures

#	Filename	Category	Purpose
1	fig_price_coverage.png	Data diagnostic	Shows final asset coverage after missing-data filtering
2	fig_missing_asset_drop_summary.png	Data diagnostic	Shows which assets were dropped and replaced
3	fig_return_distribution_all.png	Data diagnostic	Distribution of daily log returns across all assets
4	fig_asset_volatility_distribution.png	Data diagnostic	Shows cross-sectional variation in asset volatility
5	fig_correlation_heatmap.png	Data diagnostic	Visualizes cross-sectional correlation structure

#	Filename	Category	Purpose
6	fig_average_correlation_over_time.png	Data diagnostic	Tracks market-wide correlation through time
7	fig_market_return_timeline.png	Data diagnostic	Shows equal-weighted market return over time
8	fig_rolling_volatility_timeline.png	Regime diagnostic	Shows 20-day rolling market volatility
9	fig_regime_labels_timeline.png	Regime diagnostic	Marks low, medium, high volatility regimes
10	fig_covid_stress_window_timeline.png	COVID diagnostic	Highlights COVID-stress window on return timeline
11	fig_pca_explained_variance.png	PCA diagnostic	Cumulative explained variance vs number of components
12	fig_pca_scree_plot.png	PCA diagnostic	Individual explained variance by component
13	fig_pca_top_component_loadings.png	PCA diagnostic	Interprets first principal component exposure
14	fig_pca_2d_projection.png	PCA diagnostic	Two-dimensional visualization of market states
15	fig_raw_distance_distribution.png	Raw geometry	Distribution of pairwise distances in raw space
16	fig_distortion_vs_k_main.png	Main metric	Distance distortion vs k for core methods
17	fig_distortion_boxplot_by_method.png	Main metric	Distribution of distance distortion by method
18	fig_distortion_dense_vs_sparse_jl.png	JL comparison	Compares dense Gaussian JL against sparse JL
19	fig_distortion_sparse_sweep.png	JL grid	Shows sparse JL distortion across k and s
20	fig_nn5_overlap_vs_k.png	Main metric	Top-5 nearest-neighbor preservation vs k
21	fig_nn10_overlap_vs_k.png	Main metric	Top-10 nearest-neighbor preservation vs k
22	fig_nn_heatmap_method_k.png	Main metric	Heatmap of nearest-neighbor overlap by method and k
23	fig_clustering_ari_vs_k.png	Main metric	Clustering ARI vs k
24	fig_clustering_ari_by_clusters.png	Main metric	ARI sensitivity to cluster count
25	fig_cluster_embedding_compact.png	Clustering diagnostic	Visual comparison of raw/PCA/JL clusters
26	fig_anomaly_recall_vs_k.png	Main metric	Top-5% anomaly recall vs k
27	fig_anomaly_score_correlation.png	Main metric	Correlation between raw and compressed anomaly scores
28	fig_top10_anomaly_rank_comparison.png	Main metric	Rank preservation of top-10 unusual days
29	fig_runtime_transform_vs_k.png	Runtime	Transform runtime vs k
30	fig_projection_memory_vs_k.png	Runtime	Projection matrix memory footprint vs k

#	Filename	Category	Purpose
31	fig_sparse_nnz_vs_dense_nnz.png	Runtime	Nonzero count comparison for dense vs sparse JL
32	fig_runtime_accuracy_frontier.png	Runtime	Accuracy-runtime tradeoff frontier
33	fig_full_including_covid_distortion.png	COVID	Distance distortion including COVID
34	fig_excluding_covid_distortion.png	COVID	Distance distortion excluding COVID
35	fig_covid_only_distortion.png	COVID	Distance distortion only during COVID-stress window
36	fig_covid_nn_overlap.png	COVID	Nearest-neighbor preservation during COVID-stress window
37	fig_covid_anomaly_recall.png	COVID	Anomaly recall during COVID-stress window
38	fig_covid_top10_unusual_days.png	COVID	Top-10 unusual COVID days under raw and compressed spaces
39	fig_covid_vs_noncovid_metric_difference.png	COVID	Difference between COVID and non-COVID performance
40	fig_regime_distortion_comparison.png	Regime	Distortion across low/medium/high volatility regimes
41	fig_regime_nn_comparison.png	Regime	Nearest-neighbor overlap by volatility regime
42	fig_regime_anomaly_comparison.png	Regime	Anomaly recall by volatility regime
43	fig_synthetic_distortion_rank_synthetic_rankmap.png	Synthetic	Distance distortion by true rank and noise
44	fig_synthetic_nn_rank_noise_synthetic.png	Synthetic	NN overlap by true rank and noise
45	fig_synthetic_pca_advantage_synthetic.png	Synthetic	Where PCA beats sparse JL most clearly
46	fig_synthetic_sparsejl_competition_synthetic.png	Synthetic	Regions where sparse JL approaches PCA
47	fig_synthetic_noise_sensitivity_synthetic.png	Synthetic	Performance degradation as noise increases
48	fig_synthetic_rank_sensitivity_synthetic.png	Synthetic	Performance degradation as true rank increases
49	fig_jl_seed_variance.png	Robustness	Variance of JL metrics across random seeds
50	fig_best_method_by_metric.png	Summary	Best method for each metric and k

11.4 Main-Report vs. Appendix Figures

The final written report should not place all 50 figures in the main body. The recommended split is:

- **Main report:** 10–15 most important figures.
- **Appendix:** remaining diagnostic and sensitivity figures.

The main report should prioritize:

1. PCA explained variance.
2. Main distortion vs k .
3. NN overlap vs k .
4. Anomaly recall vs k .
5. Runtime-accuracy frontier.
6. COVID-only distortion.
7. COVID-only anomaly recall.
8. Synthetic PCA advantage heatmap.
9. Synthetic sparse JL competitive region.
10. Regime-specific comparison.

12 Expected Results and Interpretation

12.1 Expected Result 1: PCA Dominates Under Strong Low-Rank Structure

If financial returns have strong low-rank factor structure, PCA should perform well at small k . This would likely appear as:

- high explained variance at low k ,
- strong nearest-neighbor preservation,
- high clustering stability,
- good anomaly preservation.

This would support the view that data-adaptive compression is superior when the structure is strongly low-rank.

12.2 Expected Result 2: Sparse JL Becomes Competitive as k Increases

Sparse JL may perform poorly at very small k but improve as k increases. If sparse JL approaches dense Gaussian JL or PCA at moderate dimensions, then it may be attractive because of its computational efficiency.

This would support the claim that sparse JL can be a practical compression tool even when it is not the best purely statistical method.

12.3 Expected Result 3: Noise Weakens PCA’s Advantage

In synthetic data, when noise level σ increases, the benefit of PCA may decrease. PCA is optimal for capturing variance, but if much of the variance is noise, its advantage may become less meaningful for preserving neighborhoods or anomaly structure.

12.4 Expected Result 4: COVID and High-Volatility Regimes May Be Harder to Preserve

During high-volatility periods, market structure may change quickly. Compression methods may struggle more in these regimes. The report should examine whether:

- PCA remains robust,
- sparse JL degrades sharply,
- anomaly recall differs across COVID and non-COVID periods,
- nearest-neighbor structure is less stable during the COVID-stress window.

13 Implementation Output Contract

The final agent implementation should produce a reproducible project with the following structure.

```
project_root/
  README.md
  requirements.txt
  pyproject.toml                      # optional but preferred

  configs/
    experiment_config.yaml           # records pinned Kaggle version
    figure_catalog.yaml

  data/
    raw/
      sp500_prices.csv               # pinned Kaggle dump
    processed/
      returns_matrix.csv
      standardized_returns_train.csv
      standardized_returns_test.csv
      asset_list.csv
      date_index.csv
      preprocessing_report.json

  src/
    data_loader.py                   # loads Kaggle CSV, yfinance fallback
    preprocessing.py
    projections.py
    pca_baseline.py
    metrics.py
    synthetic_data.py
    experiment_config.py
    experiment_runner.py
    parallel_backend.py
```

```

    experiments_real.py
    experiments_synthetic.py
    experiments_covid.py
    experiments_regime.py
    plotting.py
    utils.py

scripts/
    run_data_pipeline.py
    run_real_experiments.py
    run_synthetic_experiments.py
    run_covid_experiments.py
    run_regime_experiments.py
    generate_figures.py
    run_all.py

notebooks/
    exploratory_analysis.ipynb
    final_results.ipynb

results/
    real_distance_distortion.csv
    real_nearest_neighbor_overlap.csv
    real_clustering_stability.csv
    real_anomaly_preservation.csv
    real_runtime_memory.csv
    jl_grid_search_results.csv
    synthetic_results.csv
    covid_results.csv
    regime_results.csv

figures/
    fig_*.png

report/
    final_report.tex
    references.bib

```

13.1 Minimum Required Tables

The final report should include the following tables:

1. Dataset summary table (including pinned Kaggle version).
2. Missing-data drop-and-replacement table.
3. Preprocessing summary table.
4. Method comparison table.

5. Main real-data metric table.
6. JL grid-search summary table.
7. COVID vs. non-COVID metric table.
8. Synthetic factor-model result table.
9. Runtime and memory comparison table.
10. Final recommendation table.

13.2 Minimum Required Figures

The implementation must generate at least 40 figures, following the figure catalogue in Section 11. The report should include the strongest figures in the main body and place the remaining figures in the appendix.

14 Implementation Phases

14.1 Phase 1: Finalize Design

Before coding begins, the team must finalize:

- asset universe (S&P 500 today’s constituents),
- reserve universe for replacement assets,
- data source (Kaggle dataset name and version pin),
- date range,
- strict no-imputation missing-data policy,
- core target-dimension grid,
- sparse JL sparsity grid,
- random seed count,
- COVID stress window,
- anomaly definition,
- Ray usage policy,
- exact deliverables.

14.2 Phase 2: Data Pipeline

Tasks:

1. Download adjusted close prices from the pinned Kaggle dataset (yfinance fallback if unavailable).
2. Record the Kaggle dataset version and download date in `configs/experiment_config.yaml`.

3. Build a clean price matrix.
4. Drop assets with missing values.
5. Replace dropped assets from reserve universe if needed.
6. Confirm that final price matrix has no missing values.
7. Run validation cross-check on a sample of tickers against a second source.
8. Compute daily log returns.
9. Split chronologically into train and test.
10. Fit standardization parameters on train.
11. Transform train and test.
12. Save all processed artifacts.

14.3 Phase 3: Compression Methods

Tasks:

1. Implement raw baseline.
2. Implement dense Gaussian JL.
3. Implement sparse JL with core sparsity grid.
4. Implement PCA / truncated SVD.
5. Ensure all methods expose a common interface.

The common interface should behave conceptually as:

```
fit(X_train)
transform(X)
fit_transform(X_train)
```

For random projection methods, `fit` generates and stores the projection matrix.

14.4 Phase 4: Metrics

Tasks:

1. Implement pairwise distance distortion.
2. Implement nearest-neighbor overlap.
3. Implement clustering ARI.
4. Implement anomaly recall.
5. Implement anomaly-score correlation.
6. Implement top-10 unusual day rank preservation.
7. Implement runtime and memory logging.

14.5 Phase 5: Parallel Experiment Runner

Tasks:

1. Define `ExperimentConfig`.
2. Generate experiment grid.
3. Implement local backend.
4. Implement Ray backend.
5. Ensure both backends return identical result schemas.
6. Add deterministic random seed handling.
7. Save all results centrally after worker completion.

14.6 Phase 6: Real-Data Experiments

For each method and each target dimension k :

1. Fit method on train if needed.
2. Transform test data.
3. Compute all metrics.
4. Save results.

For JL methods, repeat across 10 random seeds and aggregate mean and standard deviation.

14.7 Phase 7: COVID Experiments

Run the same metric suite for:

1. full sample including COVID,
2. full sample excluding COVID,
3. COVID-stress-only window,
4. optional COVID-year diagnostic window.

The COVID diagnostic protocol should also fit PCA and standardization using pre-COVID data only when performing the out-of-sample COVID stress analysis.

14.8 Phase 8: Synthetic Experiments

For each synthetic rank r , noise level σ , method, and dimension k :

1. Generate synthetic data.
2. Split into train and test.

3. Standardize using train statistics.
4. Apply compression methods.
5. Compute metrics.
6. Save results.

14.9 Phase 9: Figure Generation

Tasks:

1. Read saved result CSV files.
2. Generate all required figures in the figure catalogue.
3. Save figures to `figures/`.
4. Save any figure-specific source data if needed.
5. Produce a figure manifest file.

14.10 Phase 10: Analysis and Report

Tasks:

1. Generate all required tables.
2. Interpret results around the low-rank factor structure question.
3. Discuss whether sparse JL is competitive with PCA.
4. Discuss computational tradeoffs.
5. Discuss COVID-specific findings.
6. Discuss synthetic factor-model findings.
7. Write limitations (including survivorship bias from today's S&P 500 backfill).
8. Prepare final report.

15 Risks and Mitigations

Table 4: Risks and Mitigation Strategies

Risk	Why It Matters	Mitigation
Missing data across assets	Kaggle CSV may have gaps for tickers added or removed mid-period	Drop those tickers and replace from a reserve list of next-ranked S&P 500 names; document drops in <code>preprocessing_report.json</code>

Risk	Why It Matters	Mitigation
Survivorship bias (today's S&P 500 back-filled)	Companies in the 2014 S&P 500 that were later acquired, bankrupt, or demoted (e.g. Time Warner, Sprint, Allergan) are excluded; results may be biased toward calm-regime survivors	State explicitly in Section 14 (Limitations) and final report; project is about compression geometry, not trading P&L; trade-off accepted in lieu of paid CRSP/WRDS access
Kaggle dataset quality	Community-maintained data may have silent adjustment errors or stale rows	Pin a specific dataset version; cross-check a random sample of tickers against yfinance or Stooq; document discrepancies
Kaggle dataset unavailable at run time	Dataset can be removed or re-named by the maintainer	Implement yfinance fallback that loads the same fixed ticker list and produces an identical schema
No-imputation policy reduces universe size	Strict filtering may remove many assets	Use reserve list and target a realistic final N ; report final N alongside drop count
PCA data leakage	Fitting PCA on all data would inflate test performance	Fit PCA only on training data
Standardization leakage	Using full-sample mean/std leaks test information	Compute standardization parameters on train only
COVID leakage	COVID may fall inside training period under 70/30 split	Add pre-COVID diagnostic protocol
Randomness in JL	Single-seed results may be unstable	Use 10 seeds and report mean/std
Too many experimental branches	Project may become too broad	Restrict to core grid only; extras go to optional appendix
Weak real-data conclusions	Real data may be noisy or ambiguous	Use synthetic factor-model experiment to isolate low-rank effects
Anomaly definition too vague	Results may be hard to defend	Define unusual days as top 5% by raw market-state norm
Runtime results inconsistent	Machine-dependent measurements can vary	Use relative comparisons and average repeated runs
Ray overhead	Parallelization may not help small grids	Keep local backend; use Ray when grid is large
Too many figures	Report may become cluttered	Put only strongest figures in main report; move rest to appendix

16 Team Decisions Required Before Implementation

The team should approve or modify the following items before the plan is converted into an execution prompt.

Table 5: Pre-Implementation Decision Checklist

Decision	Recommended Choice
Asset universe	S&P 500 today’s constituents (~500 names); survivorship-biased
Data source	Kaggle: <code>andrewmvd/sp-500-stocks</code> (Larxel), pinned dataset version; yfinance fallback
Validation cross-check	Compare 5–10 random tickers against yfinance or Stooq for sanity
Ranking metric	Average daily dollar volume over the 2014–2024 window
Selection slices	<code>top_100</code> , <code>top_200</code> , or <code>all_survivors</code> (chosen via <code>configs/experiment-config.yaml</code>)
Stage 1 filters	Drop tickers with NaN inside window, delisted before 2024-12-31, <95% trading-day coverage, or implausible single-day returns ($ r > 0.5$ without known corporate action)
Stage 2 filters	Drop tickers with > 10 consecutive zero-return days or constant adjusted close for > 5 consecutive days
Liquidity floor	None applied (ranking already orders by liquidity)
Reserve replacement	None (ranking framework makes “top- N ” well-defined regardless of survivor count)
Date range	2014-01-01 to 2024-12-31
Return definition	Daily log returns
Missing data policy	No imputation; tickers with missing values dropped at Stage 1
Standardization	Z-score by asset using train statistics
Train/test split	Chronological 70/30
COVID stress window	2020-02-19 to 2020-04-30
COVID diagnostic training	Pre-COVID data ending 2019-12-31
Core methods	Raw, Gaussian JL, sparse JL, PCA
Core target dimensions	5, 10, 20, 30, 50, 100
Core sparse JL sparsity	$s = 1, 3, 5$
Primary sparse JL value	$s = 3$
JL random seeds	10
Anomaly definition	Top 5% by raw market-state norm
Synthetic ranks	3, 5, 10, 20
Synthetic noise levels	0.1, 0.5, 1.0
Parallelization	Ray optional but recommended for large grids
Minimum figures	At least 40 generated figures
Optional extension	Fama–French factor comparison only if time permits

17 Proposed Final Report Structure

The final project report should use the following structure:

1. Introduction

- Motivation.
- Connection to sparse JL.
- Financial low-rank structure.

2. Primary Paper Summary

- What the SOSA paper studies.
- Why sparse JL is important.
- Why its ideas motivate this project.

3. Research Questions

- Generalization of sparse JL.
- Low-rank factor structure vs. PCA.
- Role of market regimes and COVID stress.

4. Data

- Asset universe (S&P 500).
- Data source (Kaggle, pinned version).
- Date range.
- Return construction.
- Strict no-imputation cleaning.
- Standardization.

5. Methods

- Raw baseline.
- Dense Gaussian JL.
- Sparse JL.
- PCA / truncated SVD.

6. Metrics

- Distance distortion.
- Nearest-neighbor preservation.
- Clustering stability.
- Anomaly recall.
- Runtime and sparsity.

7. Parallel Implementation

- Experiment grid.

- Local vs. Ray backend.
- Reproducibility.

8. **Real-Data Results**

- Main metric comparisons.
- PCA explained variance.
- JL grid-search findings.
- Regime-specific analysis.

9. **COVID-Stress Results**

- Including-COVID vs excluding-COVID vs COVID-only.
- COVID anomaly preservation.
- COVID nearest-neighbor degradation.

10. **Synthetic Factor-Model Results**

- Rank/noise grid.
- When PCA dominates.
- When sparse JL becomes competitive.

11. **Discussion**

- Interpretation around low-rank structure.
- Computational tradeoffs.
- Relationship to the SOSA paper.

12. **Limitations**

- Survivorship bias from today's S&P 500 backfill.
- Kaggle dataset quality assumptions.
- Simplified anomaly definition.
- Synthetic model simplifications.
- Runtime machine-dependence.

13. **Conclusion**

- Final answer to the research question.
- Practical guidance on sparse JL vs. PCA.

14. **Appendix**

- Full figure catalogue.
- Full metric tables.
- Additional COVID-year diagnostic.

18 Definition of Success

The implementation will be considered successful if it produces:

1. A reproducible data pipeline anchored to a pinned Kaggle dataset.
2. A final clean return matrix with no missing values.
3. A clean comparison of raw, Gaussian JL, sparse JL, and PCA.
4. Metrics across all core target dimensions.
5. Seed-averaged JL results.
6. COVID including/excluding/only result sets.
7. Synthetic factor-model experiments varying rank and noise.
8. Ray-compatible parallel execution with local fallback.
9. At least 40 generated figures.
10. Tables and figures suitable for the final report.
11. A clear conclusion about when sparse JL is competitive with PCA and when it is not.

The strongest possible outcome is not necessarily that sparse JL beats PCA. A strong and honest result could be:

PCA dominates when the data has strong low-rank factor structure, especially at small target dimensions. However, sparse JL becomes competitive at moderate dimensions or under noisier conditions, while offering lower computational cost than dense projections. COVID-stress periods and high-volatility regimes expose where compression methods degrade. This clarifies the practical boundary between data-adaptive low-rank compression and data-independent sparse random projection in structured financial data.

19 Reference

Michael B. Cohen, Thathachar S. Jayram, and Jelani Nelson. *Simple Analyses of the Sparse Johnson–Lindenstrauss Transform*. In *Proceedings of the 1st Symposium on Simplicity in Algorithms (SOSA)*, 2018.